Sharing State

Hooks and useState work cleanly when the state is completely encapsulated inside one component, but often components have to share state. Sometimes state is shared among just one subtree of DOM elements on the page, but sometimes state is shared across a page. I'll talk about more complex state patterns in Chapter 11, *Managing State in React*, but here we will cover the simplest version of one common scenario where state is shared among parent and child components.

In this React pattern, when there is a common parent to all the components that need that state, the state is typically owned by the common parent. The parent sends the raw values down to the child components as props and typically also sends a function that the child components should call to change the parent state. When the child components call that function, the parent state changes. Then the parent component re-renders, causing the child components to re-render with new prop values. This pattern is often called "lifting state up" and "passing state down."

As an example, let's imagine we can use a pull-down menu to determine the number of seats we want to purchase at a time and that clicking on a seat also selects seats to the right of the seat clicked on such that a matching number of seats is purchased. If that number of seats is not available to the right, then the seat is marked unavailable and clicking it has no effect.

This feature adds one new piece of state: the number of seats being purchased via the pull-down menu. It also requires that Seat components need to know something about the other seats in their row to determine their status. In the first case, the state for the number of seats being purchased should be owned by the common parent of all the components that use it, which here would be Venue. Similarly, we need to lift the state of an individual seat up to the Row so that the Row can determine which seats are valid for purchase given the number of seats desired.